

HireSignal - Setup & Demo Guide

Use this guide to run the project locally and present a clean end-to-end demo

1. How to set up HireSignal on your local machine

Project prerequisites

- Windows/macOS/Linux
- Node.js 18+
- Python 3.11+
- Docker Desktop (recommended for PostgreSQL)
- Git

Step 1 - Open the project

Extract the final HireSignal project and open the project root in VS Code. Example on Windows:

D:\Development\Projects\HireSignal

Step 2 - Install frontend dependencies

npm install

npm run lint

npm run build

Start the frontend with:

npm run dev

Open <http://localhost:3000>

Step 3 - Create and activate the Python environment

cd fastapi_service

python -m venv .venv

.venv\Scripts\Activate.ps1

pip install -r requirements.txt

On Windows PowerShell, activation should change the prompt to start with (.venv).

Step 4 - Start PostgreSQL and FastAPI

`docker compose up --build`

- FastAPI: <http://localhost:8000>
- Swagger API docs: <http://localhost:8000/docs>
- Health check: <http://localhost:8000/health>

Step 5 - Run the full application

- Terminal 1: from the project root, run `npm run dev`.
- Terminal 2: from `fastapi_service` with the virtual environment active, run `docker compose up --build`.
- Keep both terminals running during the demo.

Configuration note

HireSignal core scoring is local. No Gemini/OpenAI API key is required. The frontend defaults to <http://localhost:8000>, so no `.env` file is required for normal local use. Do not add or commit secrets.

2. How to show a strong demo of HireSignal

Demo goal

Show that HireSignal is not just a UI mockup. The strongest demo proves the complete path: uploaded resume -> FastAPI file extraction -> NLP/skill matching -> TF-IDF fit scoring -> ATS red flags -> classifier output -> explainable report.

Demo setup - keep these ready

- Frontend open at <http://localhost:3000>.
- Backend running at <http://localhost:8000>.
- This demo kit available locally.
- Use the PDF versions of the resumes for the cleanest upload demonstration.

Demo sequence A - quick product tour (2-3 minutes)

- 1. Start on Overview. Explain in one sentence: "HireSignal scores how well a resume fits a job description and explains why."
- 2. Open Analyze / Scorer. Point out the candidate and job selection controls.
- 3. Run the strong-fit scenario: Priya Sharma + JD_01 Senior Backend Engineer.
- 4. Show the Fit Score, matched skills, and missing skills. Explain that the result is generated by the backend scoring pipeline.
- 5. Point out the ATS red-flag panel and the classifier labels rather than reading every item.
- 6. Open System. Walk through extraction -> skills -> TF-IDF -> red flags -> classifier -> PostgreSQL.

- 7. Open Benchmarks. Mention that the included classifier artifact has evaluation metrics and that these are not production guarantees.

Demo sequence B - prove real PDF/DOCX upload (2 minutes)

- 1. In Analyze / Scorer, switch to custom/upload resume mode.
- 2. Upload Priya_Sharma_Backend_Engineer.pdf (or the DOCX file).
- 3. Paste the JD from JD_01_Senior_Backend_Engineer.txt into the custom JD box if needed.
- 4. Run the analysis.
- 5. Explain: "This file is sent to POST /score-upload, parsed by Python, then scored by the same backend pipeline."
- 6. Show the browser result and, if the audience is technical, open /docs to show the actual FastAPI API contract.

Demo sequence C - show the red-flag detector

- 1. Upload Arjun_Mehta_ML_Engineer_ATS_Test.docx.
- 2. Compare it against JD_02_Machine_Learning_Engineer.txt.
- 3. Show that the role fit can still be reasonable while the resume receives ATS warnings.
- 4. Point out the intentional issues in this synthetic document: a generic objective, repeated keywords, a career-break timeline, and weak contact/profile formatting.
- 5. Explain that this demonstrates the difference between "technical fit" and "resume quality / ATS hygiene."

Demo sequence D - show model comparison (optional)

- 1. Use Priya + JD_01 for the backend-heavy scenario.
- 2. Use Arjun + JD_02 for the NLP/ML scenario.
- 3. Compare matched skills, missing skills, red flags, and classifier predictions.
- 4. Emphasize that a higher or lower score is an evidence-based signal from the implemented heuristics and model, not a hiring decision.

Technical talking points for reviewers

- TF-IDF cosine similarity provides the lexical fit component.
- Skill overlap is computed from a curated taxonomy with hundreds of technical signals.
- Rule-based checks detect 11 ATS-style red-flag categories.
- The classifier uses a One-vs-Rest logistic-regression approach and ships with an evaluation artifact.
- FastAPI exposes JSON scoring and multipart file-upload endpoints with rate limiting.
- PostgreSQL schema and Alembic migration demonstrate persistence/database design even when the demo focus is the scoring workflow.
- The project intentionally avoids a paid external LLM dependency for its core scoring pipeline.

What not to claim during the demo

- Do not say the system "understands" resumes like a human recruiter. It combines lexical similarity, skill matching, rules, and a classifier.

- Do not claim that the score predicts hiring outcomes.
- Do not claim the classifier is production-validated; the README correctly frames the included benchmark as artifact-level evaluation.
- Do not imply that the frontend performs the real file parsing. PDF/DOCX upload parsing happens in FastAPI.

Recommended one-line opening

"HireSignal is an explainable resume-to-job matching system that turns an uploaded resume and job description into a fit score, skill evidence, ATS warnings, and classifier signals through a FastAPI-backed ML pipeline."